Code generation from scratch

HW2 experiment report - work in progress
Andrei Vorsin | 16 October 2026

Current result

Verified and scored checkpoints reach 180,000 updates. Ten initial checkpoints, eight first-continuation checkpoints and 6 longer-continuation checkpoints were saved in Google Drive and scored by the course bot. The best public result is **57.08% (548/960)**. The bot reported rank 8 of 15 at 2026-10-16T08:59:00+03:00. Its exact scored output is vote-180k-strict-majority.jsonl. This is the best historical result across decoding experiments. The user now selects strict greedy decoding; its separate results are listed at the end. T4 timing remains deferred. No private result is available.

Task and rules

Train an encoder-decoder Transformer from random weights using only the supplied SlavaYus/tinysms_qa_12m training split. Generate one Python program per test question. All 1,919 programs must be generated within 180 seconds on one NVIDIA T4. No pretrained weights, outside examples, model distillation, or agent-written target programs are used.

Data and architecture

The allowed dataset contains 11,838,003 question/code pairs in 60 training files. The test file contains questions only. A byte-level BPE tokenizer with 8,192 tokens was fitted on shard 0 after reserving its first 1,024 rows. The saved model has 9,967,360 parameters: four encoder layers, four decoder layers, width 256, feed-forward width 768, four attention heads, gated GELU and dropout 0.1. It uses the T5 architecture with randomly initialized weights.

Training uses AdamW, 500 warmup updates, cosine decay to 10% of the initial rate, weight decay 0.01 and gradient clipping at 1.0. The first 20,000 updates used batch 64 and peak learning rate 0.001. The first continuation retained optimizer moments, used batch 128 and restarted the schedule at peak rate 0.0005. The 60k-to-180k continuation retains batch 128 and optimizer moments, with a new schedule peaking at 0.0003. Input and target limits are 128 and 192 tokens. Padding tokens do not contribute to the loss. Model and optimizer states are saved regularly.

Validation correction

Earlier runs excluded the first 1,024 row positions but did not exclude copies of their questions. An audit found 24 later rows in shard 0 with the same question text. Historical validation was therefore not fully independent. Future streams exclude every reserved question text across all shards. Rows 0-255 select checkpoints; rows 256-511 provide separate diagnostics. The earlier tokenizer may have seen duplicate question text.

Quality experiments

Historical runs

Experiment	Evidence	Interpretation
Local MPS / short T4 checks	One local update; ten T4 updates; later T4 pilot reached 4,000 updates, loss 1.6469.	Pipeline checks. The longer pilot checkpoint was not recovered.
Sequential shards, 20,000 updates	Final held-out token loss 1.1213; best logged 0.9207 at step 12,500.	Best weights were not retained; last weights only.
Rotate after 1,000 mini-batches, 20,000 updates	Final loss 0.9815; saved best 0.8636 at step 18,500.	Selected historical checkpoint. Held-out exact code match 2/256; public score 18.85%.

The historical comparison supports broader data coverage by token loss, but the validation-copy issue limits that conclusion. Token loss scores the next token with the reference prefix supplied; it does not show whether a generated program computes the right answer.

Audit before the new pilots

All 1,024 held-out programs round-trip through the tokenizer exactly with cleanup disabled. Only 2 held-out questions and 3 of 1,919 test questions exceed the input limit; 5 held-out programs exceed the target limit. Truncation and tokenizer corruption do not explain most failures in this audit. Samples of 1,000 rows from shards 0, 15, 30, 45 and 55 have mean code lengths 65.96, 62.56, 93.17, 61.84 and 118.95 tokens. This suggests that sequential training encounters different styles for extended periods.

Matched sampling pilot

Hypothesis: mixing original examples across shards improves held-out numeric agreement. Two fresh runs use identical initialization, model, seed, BF16 precision, batch 64, accumulation 1, grouping window 32 and 6,000 updates. Both exclude validation question texts. The control rotates after 1,000 batches. The treatment mixes up to 4,096 randomly selected original rows from every shard and resamples each epoch. Sampling is the experimental variable. Comparisons with older runs also change batch shape and validation exclusion.

Run	Best token loss	Held-out numeric agreement
control	1.6012	14/245
mixed	1.4793	16/245
mixed_lr001	1.1409	27/245

Numeric evaluation compares solve() outputs of generated and supplied held-out programs, using limited separate processes and restricted imports. Only finite numeric reference results are usable. The old saved model agrees on 50/245 usable references (20.4%). Reference errors and runner restrictions can affect this measure. Numeric outputs are never used as training targets.

Completed Drive-backed run

All ten checkpoints retained

The mixed sampler completed 20,000 updates with batch 64, BF16 training and a peak learning rate of 0.001. Source commit: 1a23e45b1138c3dba7c40befeae6d3337153df98. Training plus periodic validation, generation and Drive saves took about 12 minutes on an NVIDIA RTX PRO 6000 Blackwell Server Edition. This is not a T4 timing measurement.

Step	Validation token loss	Public correct / 960	Public score
2,000	1.8064	31	3.23%
4,000	1.2130	49	5.10%
6,000	1.0967	93	9.69%
8,000	1.0020	103	10.73%
10,000	0.9396	141	14.69%
12,000	0.8848	133	13.85%
14,000	0.8540	163	16.98%
16,000	0.8219	182	18.96%
18,000	0.8076	192	20.00%
20,000	0.7952	195	20.31%

Every checkpoint was written to Drive with optimizer state and a completion manifest before predictions were generated. Each checkpoint has 1,919 exact model outputs, matching CSV and JSONL files, validation metrics and provenance hashes. The files were exported from Drive, checked locally and then submitted to the bot. Colab was disconnected before the bot evaluations finished.

Drive location: MyDrive/HSE_NLP_HW2/runs/mixed-lr001-20k-drive-v1. The repository keeps exported prediction evidence and individual submission receipts. Weights stay in Drive. The final validation loss is lower than in the short pilots, but scores across checkpoints show that lower token loss does not guarantee higher program accuracy.

Longer training and GPU use

Eight more checkpoints, all scored

The continuation added 40,000 updates and 5,120,000 example presentations in 1,957.2 seconds (32.6 minutes), including validation, generation and Drive saves. It resumed our own step-20,000 model. Source commit: a411a64f8afe61e3a2b337af338c0ea6c19c9c15. No new target programs or external model weights were introduced.

Step	Validation token loss	Public correct / 960	Public score
25,000	0.7647	194	20.21%
30,000	0.7120	227	23.65%
35,000	0.6817	231	24.06%
40,000	0.6451	273	28.44%
45,000	0.6058	314	32.71%
50,000	0.5857	343	35.73%
55,000	0.5701	353	36.77%
60,000	0.5652	362	37.71%

Batch size was chosen from two matched 16,384-example probes with reversed order. Median throughput was 2,100 examples/s at batch 64; 2,279 at 128; 2,114 at 256; and 1,950 at 512. Batch 128 was 8.5% faster than 64, with peak allocated memory 7.75 GiB. Larger batches raised utilization but reduced useful throughput.

Across the continuation, 1,972 one-second GPU samples averaged 60.38% utilization, with median 78%, 16.23% zero samples and peak used memory 10,397 MiB. These include checkpoint saving and generation. CPU preparation occupied about 41% of the batch-128 probe, suggesting a remaining bottleneck. The GPU was released and Colab showed no active sessions at 19:07 +03:00.

Drive location: MyDrive/HSE_NLP_HW2/runs/continue-20k-60k-b128-v1. Every checkpoint has exact 1,919-row outputs in official test order, provenance hashes and a bot receipt. The step-60,000 Blackwell generation took 9.07 seconds; its T4 timing remains unverified.

Faster input preparation

Matched CPU and GPU checks

CPU preparation compared the original batch path, reuse of window tokenization, and a bounded two-batch preparation queue. All three produced identical input IDs, masks and labels for 32,768 examples and 2,419,826 target tokens. Preparation took 18.72, 6.38 and 5.98 seconds respectively. Eighteen local checks covered truncation, duplicates, partial windows, literal special tokens and producer failures. Synthetic fixtures stayed confined to tests.

Input path	Median examples / second
legacy	2,931.17
encoded	3,483.24
encoded-prefetch	3,692.80

Two GPU rounds used reversed variant order, the same checkpoint, seed, examples, precision and batch size. The selected two-batch queue improved median throughput by 25.98% over the original path, with peak allocated tensor memory 7.75 GiB. The tokenizer used by the producer is separate from the validation tokenizer. This is a matched speed result; utilization alone does not measure useful throughput.

Longer continuation: 60k to 180k

The run resumes our own verified 60,000-step checkpoint and schedules 120,000 additional updates (15,360,000 example presentations). It keeps the mixed sampler, batch 128 and BF16 training. Source commit: ca26869455692cb570b94cb4a633be7701221f9e. Weights and exact predictions are saved to Drive every 20,000 updates before export and bot evaluation.

Step	Validation token loss	Public correct / 960	Public score
80,000	0.5250	414	43.13%
100,000	0.4909	455	47.40%
120,000	0.4712	472	49.17%
140,000	0.4528	487	50.73%
160,000	0.4419	516	53.75%
180,000	0.4402	517	53.85%

The saved completion record reports return code 0 and 6 checkpoints. All six were scored. The last 20,000 updates added one public correct answer (516 to 517). Training was stopped at its planned endpoint, GPU release was verified, and subsequent file work ran on CPU.

Each export is checked against its Drive manifest, source commit, checkpoint hash, official question-order hash and exact decoded selected token IDs. Bot receipts are written and read back on Drive. The current-generation measurements use Blackwell and do not establish T4 compliance.

Generation and speed

Measured optimization experiments

The course materials motivated length grouping, mixed precision, gradient accumulation, clipping and caching. The model already uses cached autoregressive generation. Training disables that cache. Loss accumulation stays on the GPU until logging. These choices were measured rather than assumed to make the full workload faster.

Blackwell inference, same weights	Median seconds	Changed outputs
Before, FP32	11.350	0
After, FP32	11.240	0
After, FP16	13.027	54
After, BF16	12.249	313

Two repetitions used reversed variant order on the second pass. The approximately 1% FP32 difference has overlapping timing ranges; it is not convincing evidence of a speedup. Lower inference precision was slower and changed programs. FP32 remains the default. An 80-update continuation benchmark also found no clear speed or validation-loss improvement; it does not establish full retraining quality.

T4 requirement

The historical 39.09-second T4 measurement belongs to an older run. Blackwell timings cannot establish the T4 requirement for the current candidate. A bounded final-checkpoint T4 experiment is reported separately below; only its measured configurations can establish compliance.

Exact model output and submission

Generation sorts questions by token length and restores their original positions. Historical runs followed greedy decoding with four-beam generation for syntactically invalid outputs. As of 16 October, strict greedy is selected: one candidate, no sampling, no syntax fallback, and no execution voting. No candidate is rewritten. Selected token IDs are saved, then decoded again to check equality with every output string. A sidecar records source commit, checkpoint hash, question-order hash, output hash, device and generation settings. JSONL conversion only wraps each exact string as a code field.

The historical submission included one handwritten identifier repair, which does not meet the user's stricter model-only condition. That repair has been removed from all future inference. The old score is retained as historical evidence, not a score for a newly verified model-only candidate.

The user confirmed that the required input is the official test.parquet question column in its original 1,919-row order. Generation preserves that order. The separate pass1_questions.jsonl is no longer a prerequisite.

Agent use and reproducibility

What the agent did

Codex read the assignment and course weeks, wrote data access, training, inference and diagnostic code, controlled Colab through MCP, inspected GPU state, compared measurements, prepared Telegram file packaging and wrote this report. It identified incomplete validation exclusion, stale T4 claims and handwritten postprocessing. Those findings changed the data exclusion rules, provenance records and speed checks.

Codex did not provide solution programs as training targets or replace model answers with its own solutions. No external model was used as a teacher. The submitted programs must come from the trained Transformer. Bot examples are not used to tune individual answers. The humanizer skill is used only to make report prose direct; it does not conceal agent involvement.

What remains open

The early pilot weights were lost when temporary runtime storage was released before export succeeded; their measurements remain saved. The revised workflow passed CPU save and restore checks and retained all ten initial, eight first-continuation and six longer-continuation checkpoints, exact predictions and bot receipts in Drive. The training GPU was released. A child-process release request failed at the notebook disconnect step; the new inference notebook executes its final release from the notebook kernel and has a hard process timeout.

Longer training exceeded the 33% public baseline, but the first-place objective remains unmet. The best candidate and all intermediate checkpoints have actual bot scores. Changed decoding outputs require a new quality comparison; speed alone does not establish correctness. A public first-place result would not establish private first place. No private result is claimed.

Reproduction files

Repository: OsVox/hse-nlp, branch 2026, homeworks/hw2. The directory contains train.py, data.py, numeric_eval.py, inference.py, tokenizer files, saved model weights, inference and training notebooks, experiment records and submission receipts. quality_experiments.md specifies the controlled pilot before its run. The raw logs and run configurations accompany the quality results.

Use a fresh output directory for each experiment. Run training with the recorded settings and keep latest plus best checkpoints. Run numeric evaluation on reserved training rows only. Use hw2_inference_colab.ipynb on a T4 to generate and package the test outputs. Verify every checkpoint in Drive before releasing compute; export predictions and record bot receipts from Drive.

Sources

Course assignment; allowed dataset and schema; course week 2 (encoder-decoder attention and decoding), week 3 (training methods), week 4 (attention and caching). Local measurements: metrics.json, gpu_benchmark/*.json, quality_results.json when present, and submissions/2026-10-15-gpu-check.json.

Alternatives to more updates

Held-out decoding and checkpoint averaging

Both experiments used reserved training questions, with no public answers or bot failure examples used for selection. The numeric runner executes supplied references and exact model outputs with time and resource limits; 245 of 256 references were usable. Agreement is a diagnostic, not the course bot score.

Experiment	Numeric agreement	Generation seconds, CPU	Decision
120k greedy	120/245	10.512	Retain
120k four beams	125/245	57.190	5.44 times slower; not adopted
140k unchanged	122/245	10.160	Retain
Average of 120k + 140k	124/245	10.313	Mixed evidence; not adopted

Averaging used identical architecture, tokenizer and training-run settings. Parent weights were verified and averaged in CPU float64, then cast back and reopened. It adds no training targets or extra inference model. Validation token loss worsened from 0.452825 to 0.459383, and execution failures rose from 16 to 22. The two-answer diagnostic gain does not justify replacing the parent.

Scaling-law estimate

The existing train-only probe contains 15,358 rows, sampled about equally across the 60 shards. With our tokenizer and length limits, mean question length is 46.91 tokens and mean code length is 73.83. Extrapolation gives approximately 555 million source tokens and 874 million target tokens per corpus. The last shard is smaller, so these are rough, non-deduplicated estimates. About 0.15% of sampled questions and 1.94% of sampled programs exceed the length limits.

Chinchilla suggests about 20 training tokens per parameter as a rough compute-efficient guide. Dividing target tokens by 20 gives 44 million parameters; counting source plus target gives 71 million. These are two accounting proxies, not a confidence interval or a fitted optimum for encoder-decoder code generation. Encoder and decoder capacity have different scaling behavior, and lower token loss need not imply higher task accuracy.

Proposed first larger model: width 512, feed-forward width 1536, six encoder layers and four decoder layers, totaling 42,481,152 parameters. Keeping four decoder layers limits repeated work during generation. At unchanged efficiency and time proportional to parameter count, one corpus-sized amount of data would take roughly 3.8-4.4 hours on the measured training GPU. This is a planning estimate; a short throughput/quality pilot and actual T4 timing must precede long training.

Hoffmann et al., Training Compute-Optimal Large Language Models; Ghorbani et al., Scaling Laws for Neural Machine Translation. Reproduction: scaling_estimate.py and experiments/scaling-analysis/.

Final-checkpoint T4 inference

Protocol

Prepare and verify model weights, tokenizer, official 1,919 questions and source on CPU. Save a hash-verified bundle to Drive, export and verify its ready record, then allocate one T4. Compare FP32 batch sizes 64, 128 and 256, twice with reversed order. Keep greedy generation, cached decoder state, length sorting, syntax-only beam retry and input/output limits 128/192 unchanged. Save every exact output, token provenance, hash and measurement to Drive. The two-model run omits optional profiling to keep GPU use bounded.

Generation time includes tokenization, sorting, generation, retries, decoding and local CSV writing. Full-call time additionally covers test reading, model hashing, provenance writing and verification. Model load and a small warm-up are measured separately. Drive archival is outside the generation timer. The experiment has a 600-second soft budget and a bounded child-process timeout; the notebook requests GPU release in finally.

Model	Batch	Median, s	Slowest, s	Same outputs as 64
9.97M, 180k	64	28.885	29.122	True
9.97M, 180k	128	16.676	16.738	True
9.97M, 180k	256	10.586	10.791	True
42.48M, 6k	64	37.032	37.585	True
42.48M, 6k	128	23.146	23.502	True
42.48M, 6k	256	20.304	20.556	True

Device: Tesla T4; PyTorch 2.11.0+cu130; Transformers 4.57.6. Model loading took 0.289 seconds and warm-up took 0.954 seconds. Changed programs require separate quality evaluation; byte-identical outputs preserve the same predictions.

Batch 256 is selected for these immutable weights. Both models finish far below 180 seconds. Every run matches its already-scored JSONL exactly: 53.85% for the established model and 18.96% for the 6k larger pilot. Duplicate bot submissions were avoided. Later weights need a fresh timing and output check.

Full runs, per-call times, memory peaks, provenance and output hashes are retained in experiments/t4-comparison-v1 and its Drive archive. Reproduce with hw2_t4_comparison_colab.ipynb; the main inference notebook now selects batch 256 for the established checkpoint. The completed T4 runtime was manually released after its asynchronous release request left it connected.

Short architecture and rate pilots

Four fresh models, 2,000 updates each

The screen kept seed 42, batch 128, BF16 training, the same mixed data stream, tokenizer and 128/192 token limits. All runs used 200 warm-up updates and a short cosine schedule. Width, encoder depth and learning rate were varied in separate comparisons. One immutable checkpoint per trial was saved in Drive, exported with hashes and selected token IDs, verified locally, then scored once by the bot. Source: 1f1558515ce00d1be29f032bac843bcbfc5b6bf4.

Parameters / layers	Peak LR	Val. loss	Held-out / 245	Public / 960	Train + export, s
9.97M, 4/4	.001	1.5262	18	66 (6.88%)	106.67
35.66M, 4/4	.001	1.9631	12	31 (3.23%)	181.02
42.48M, 6/4	.001	2.0145	9	27 (2.81%)	193.07
42.48M, 6/4	.0003	1.6312	23	86 (8.96%)	190.10

Keep the 9.97M model at .001 and the 42.48M model at .0003. The small model has lower token loss and lower cost; the larger finalist has higher held-out execution agreement, confirmed by the bot. Lowering the rate within the 42M architecture improved all quality measures. The larger high-rate models do not justify continuation from this screen.

These are early-learning results from one seed, not final convergence estimates. Even the best pilot remains below the established 180k model at 53.85%. The bot reported public rank 10/15 on 16 October; earlier rank 9/15 was an earlier snapshot. No private score is available. The later T4 comparison covers the established model and the larger 6k finalist.

Time and next comparison

The driver took 718.51 seconds after data restoration; its launcher record spans 758.79 seconds including restoration and imports. Across 716 GPU samples, mean activity was 72.90%, median 97%, and peak memory 18,627 MiB. Activity is not achieved compute throughput; startup, I/O and execution evaluation contribute low samples. Batch-size speed still needs a matched benchmark.

Continue both finalists from 2k to 6k with their optimizer moments, batch 128 and respective learning rates. Both receive the same new deterministic sample stream, a restarted 4k cosine schedule and 200 warm-up updates. Save and score the 6k checkpoint for each. Check actual T4 inference before long training. The four-way GPU was released manually after the automatic notebook request left it connected; file work resumed on CPU.

Evidence: experiments/short-pilots-v1, its selection.md and export.json, four submission receipts, finalist_pilots.py and hw2_finalist_pilots_colab.ipynb. Full weights, optimizer states and exact predictions remain in Google Drive.

Finalists after 6,000 updates

Matched continuation

Both models resumed their own 2k weights and Adam moments. Batch 128, BF16, original targets, the same new mixed sample stream, a restarted 4k cosine schedule and 200 warm-up updates were held fixed. Peak rates remained .001 for 9.97M and .0003 for 42.48M. Source: 10db3cda45ba777e04e4195f003a6b1db5254b24.

Model	Val. loss	Held-out /245	Public /960	Added train + export, s
9.97M	0.9463	38	154 (16.04%)	183.73
42.48M	0.9782	41	182 (18.96%)	353.36

The larger model leads execution accuracy in both evaluations; the smaller model has lower next-token loss and lower cost. The public difference is 28 answers (2.92 percentage points), while the held-out difference is only three answers. One seed and early checkpoints cannot establish an optimal model size or statistical significance. Both remain below the established 180k model at 53.85%.

Measured cost

The added training loop took 160.2 seconds for 9.97M and 333.3 seconds for 42.48M. The wider timings in the table include startup, validation, checkpoint saves and all-test generation. At the larger model's measured rate, a corpus-sized amount of examples would take about 2.1 hours of training loop time at batch 128, before evaluation and archival. This improves on the older 3.8-4.4-hour parameter-linear estimate but remains a short-run extrapolation.

Across 583 GPU samples, median activity was 96%, mean activity 72.15%, and peak used memory 18,631 MiB. Activity includes I/O and numeric evaluation and does not measure achieved compute efficiency. A separate batch probe uses the actual encoded/prefetched training input path, equal examples, reversed rounds and timing-boundary synchronization.

Checkpoint and evaluation retention

Both immutable 6k checkpoints, optimizer states, predictions and validation records were saved to Drive. The evidence archive was created there before download. Local checks verified hashes, all 1,919 official question positions and exact decoding of selected token IDs. Each checkpoint was submitted once to the course bot. Receipts are retained separately from the pre-submission archive. No answer text was edited.

The G4 runtime was manually released after its asynchronous release request left it connected. Standard CPU was used to stage the established 180k model and the larger 6k finalist for actual T4 inference. A faster training batch must also pass a short equal-example quality check before being adopted for full training.

Evidence: experiments/finalist-pilots-v1, its selection.md and export.json, the two 6k submission receipts, experiments/t4-comparison-v1/ready.json and benchmark_production_batches.py.

Larger model: speed and quality

The 42.48M model used one prepared G4 session for a speed probe, short quality checks and a capped continuation. All eight probe passes used 32,768 original examples and 2,419,826 target tokens; the second round reversed the batch order. Probe weights were discarded.

Batch	Median examples/s	Peak allocated GiB
64	1794.6	8.18
128	1623.4	15.78
256	1433.7	30.97
512	1319.9	61.34

Batch 64 was 10.55% faster than 128. Both quality pilots resumed the same own 6k weights and optimizer state, with fixed added example counts, BF16 and peak rate .0003. The predefined gate required no fewer correct held-out executions and token loss within 1% of the control.

Batch	Added examples	Token loss	Held-out /245	Public /960
128	262,144	0.8854	45	183
64	262,144	0.8779	41	182

Batch 128 was retained. Lower token loss at batch 64 did not compensate for its lower execution accuracy. The chosen lineage continued to 2,560,000 total example presentations, equivalent to 20,000 updates at batch 128; this is not a full pass over the 11.84M-row corpus.

Continuation step	Token loss	Public correct /960
10,000	0.8731	135
15,000	0.6995	266
20,000	0.6167	320

The 10k public regression followed a learning-rate restart; causality was not established. The final held-out result was 75/245. The grouped driver took 27.05 minutes including probes, preparation, evaluation and Drive saves. Each checkpoint retained exact token provenance and a separate bot receipt; no program was rewritten.

Decoding opportunity

On the established 180k model, four-beam decoding improved held-out numeric agreement from 131/245 to 137/245 and reduced execution failures from 19 to 11. CPU generation took 10.61 versus 50.37 seconds. The subsequent full test export scored 544/960 (56.67%), versus 517/960 (53.85%) for greedy decoding. All 1,919 questions took 503.50 seconds on CPU at batch 64, FP32. T4 timing for this output remains unverified and is deferred. Every program matches its saved model token decode; no program was rewritten.

Source: 85532a78132c37ceb9cbf3f6cfff5c478d69258f. Evidence: experiments/grouped-42m-20k-v1, experiments/decoding-180k and experiments/beam4-quality-180k-v1 (source d1aec6e). Results were downloaded with the authenticated local Drive CLI, avoiding Colab allocation for export.

Full model and decoding checks

The 42.48M model resumes its own 20k weights and Adam moments. The fixed target is 100k updates, batch 128, BF16, peak learning rate .00015, 500 warmup updates, cosine decay and the encoded/prefetched input pipeline. Halving the prior peak is a restart hypothesis, not a fitted optimum. The first 30k and 40k bot scores are review points.

Before allocating G4, CPU checks verified all dataset hashes, loaded the model and optimizer, checked finite optimizer moments and saved a verified Drive bundle. At startup the GPU showed 98% activity and 18,951 MiB used out of 97,887 MiB. This is an observed snapshot, not a utilization guarantee. Each 10k checkpoint saves weights, optimizer state, exact predictions and a separate bot receipt.

Step	Token loss	Public correct /960	Public %
20,000 (parent)	0.6167	320	33.33
30,000	0.5475	409	42.60
40,000	0.4904	471	49.06
50,000	0.4516	519	54.06

At 3.84M example presentations, the larger model scored 409/960, versus 273/960 for the smaller model at step 40k. Different learning-rate schedules and data streams prevent a causal size-only comparison. These are scored checkpoints, not a completed target run. These historical exports use greedy decoding with syntax-only beam retries. No test labels are used for training. Source: 69829cbe; evidence: experiments/full-42m-100k-v1.

Execution agreement without new training

Using the fixed 9.97M, 180k model, generate four deterministic beam candidates and execute their solve() functions in the restricted diagnostic runner. Select an unchanged program from a strict majority of all four numeric answers; otherwise retain the first executable candidate, breaking ties by model rank. References enter only after selection, for evaluation. No program is rewritten or added to training.

Selection	Rows 512:768 (247 usable)	Rows 768:1024 (246 usable)
Top beam	141	136
First executable	142	136
Strict majority	142	138

Strict majority passed the predefined positive-gain gate on both subsets: three extra correct answers across 493 usable references. This small exploratory gain is not statistical significance. First-executable selection failed the confirmation gate. Candidate and selected token decodes were checked after Drive export. The historical validation-copy caveat still applies to the 180k lineage.

Both tests ran on four CPU cores of the 48-core training host with CUDA hidden; GPU training continued. The full-test export scored 548/960 (57.08%), four more than beam-only decoding; CPU generation and selection took 279.43 seconds. T4 timing is unverified. Inspiration: Wang et al., Self-Consistency; our deterministic beam experiment differs from their sampling method. Sources: f047d7d (pilot), 124e46d (test export). Evidence: experiments/candidate-vote-180k-v1 and candidate-vote-test-180k-v1.

Selected policy: strict greedy decoding

The user selected greedy decoding on 16 October. All new evaluated exports use num_beams=1 and do_sample=False, with syntax retries and execution voting disabled. Historical results are preserved with their original settings. The running trainer retains its immutable source; a separate CPU export process applies strict greedy to its saved weights without restarting training.

Step	Token loss	Public correct /960	Public %
50,000	0.4516	519	54.06
60,000	0.4255	540	56.25

The 50k strict greedy output scored 519/960 (54.06%), matching the public score of its earlier retry-enabled output. Two of the 1,919 output strings differed; the comparison also changed device and batch size, so this is not a controlled attribution of the difference to retries alone. CPU generation took 99.67 seconds at batch 64. This is not T4 timing.

The default-decoder regression test supplies malformed Python and verifies that it is preserved with no extra generation calls. Full exports verify every program against saved token IDs, the official question-order hash, checkpoint hash and source commit. Each output is first saved to Drive, exported with the authenticated CLI, checked again and submitted once. Receipts are read back from Drive byte-for-byte.

Source: 8dcb74d2164a30c7b3169dcf7adf76114139761f. Evidence: experiments/decoding-policy.json, experiments/strict-greedy-42m-v1 and subsequent strict-greedy-42m checkpoint directories. The main inference notebook follows this policy; the earlier voting notebook is retained only for historical reproduction. First place remains unmet.